The 31 October build strategy

Mission, architecture and decision boundaries - one-glance mind map

HACKATHON TARGET

Locate simulated trapped survivors in an unknown, GPS-denied site.

01 SIMULATION

Random rooms / rubble / walls; 2 drones + 2 rovers; hidden survivor locations.

02 PERCEPTION

Local range scans + probabilistic thermal / acoustic cues; no ground truth exposed.

03 LEARNING

Frontier baseline -> shared PPO by robot type -> MAPPO only if feasible.

SYSTEM OBJECTIVE

AUTONOMOUS SWARM

2 aerial agents + 2 ground agents

local policy / peer messages / shared discoveries

04 COOPERATION

Peer messages: detections, goals, graph updates; range, latency and dropouts.

05 LOCALIZATION

Relative frames, odometry drift + confidence; shared-start alignment first.

06 ROBOTICS

ROS 2 / Gazebo replay, local decision nodes; rover nav + drone proxy.

EXECUTION LOOP

GENERATE > SENSE > SELECT GOAL > MOVE > MESSAGE > UPDATE MAP > EVALUATE

MINIMUM VIABLE DEMO

4 autonomous agents | unknown map | simulated survivor detections | peer-to-peer coordination

Compare vs frontier baseline; show loss-of-comms recovery. No claim of real thermal SLAM or 3D flight.

Build the world before the intelligence

STEP 1 creates a fair testbed; STEP 2 guarantees a working non-RL baseline

STEP 1 Python disaster world

26-27 OCT

WORLD + AGENT MODEL

- Create seeded 24x24 -> 40x40 unknown grids: rooms, corridors, impassable walls, rubble, blocked passages, safe spawn points.
- Spawn 2 drone-like + 2 rover-like agents; use 2D aerial abstraction with separate mobility rules (no flying through walls).
- Randomize 1-4 hidden survivors; keep their true coordinates strictly inside the environment.

SENSORS, OBSERVATIONS, CONTROLS

- Local ray-cast occupancy scan; short rover range and wider aerial range with line-of-sight occlusion.
- Thermal and acoustic = probabilistic detection cues with noise, misses and false positives; not real sensor emulation.
- Per-agent local map: unknown / free / blocked; local pose + simulated odometry drift. No global GPS or hidden map in policy input.
- Action = choose among <=5 local frontiers / investigate / hold. A* moves toward the chosen reachable target.

API + EXACT DELIVERABLE

reset(seed) > observe(agent) > step(actions) > render()

Implement PettingZoo ParallelEnv + NumPy world; Pygame for live visualization. Save maps, events and trajectories as JSON/CSV.

PASS GATE Run 3 seeded maps with collision-free movement and no hidden-state leakage.

STEP 2 Mapping + baseline

27-28 OCT

NON-LEARNING BENCHMARK

- Build local occupancy maps from only each agent's sensor observations; update unknown cells when observed.
- Find frontier cells (known free cells adjacent to unknown); cluster into candidate exploration targets.
- Choose nearest useful unclaimed frontier and navigate with A*; reserve goals to reduce duplicate searches.
- Investigate uncertain survivor detections; require repeat evidence or second-agent confirmation before marking located.

TOPOLOGICAL MAP, FROM LOCAL KNOWLEDGE

- Compress explored cells into graph nodes: rooms, junctions, frontiers, suspected survivor sites.
- Edges store estimated distance, traversability, observation confidence and last update time.
- Use NetworkX for graph changes and shortest-path logic; maintain distinct maps per robot.

FILES + CHECKPOINTS

`world.py / sensors.py / env.py / frontier.py / maps.py / viewer.py`

DATA BOUNDARY

World holds truth -> sensor returns noisy local evidence -> local map updates -> baseline sets goal. Policies never see truth.

PASS GATE 4 agents explore + produce maps and confirmed target markers.

Add distributed intelligence

STEP 3 learns goal selection; STEP 4 handles peer communications without a central commander

STEP 3 Cooperative RL

28-29 OCT

TRAINING DESIGN

- RLlib PPO with 2 shared policies: one for both drones, one for both rovers; each agent acts using local inputs only.
- Fixed observation vector: 9x9 local map crop + up to 5 frontier features + detector confidence + battery + received peer summary.
- Discrete policy action: select frontier slot 1-5 / investigate / hold; mask invalid choices, then let A* handle motion.
- Curriculum: one rover -> two rovers -> two drones + two rovers; train on randomized seeded maps.

REWARD + COMPARISON PROTOCOL

Illustrative team reward (tune, do not claim optimal):

```
+5 confirmed survivor  +0.1 new cell  -1 collision  -0.01 / tick
```

- Score on 20 unseen seeded maps against frontier baseline: found victims, map coverage, completion time, collisions.
- Export checkpoint + plots. If PPO fails to outperform baseline, demo baseline honestly and show training curve.

ADVANCED, ONLY AFTER BASELINE WORKS

MAPPO: centralized critic during OFFLINE training, per-agent actors at runtime. Official implementation requires environment adaptation; it is not a switch in RLlib.

PASS GATE Checkpoint runs inference with networking turned on/off.

STEP 4 Local peer network

29 OCT

REALISM IN A PYTHON NETWORK SIMULATOR

- Build sender -> neighborhood delivery -> receiver bus; no server chooses assignments or provides global state.
- Use configurable range + wall attenuation / line-of-sight, 0-30% packet loss, delay, stale-message expiry and partitions.
- Messages carry: sender ID, seq, frame ID, local pose + uncertainty, frontier graph deltas, claimed goal and survivor evidence.
- De-duplicate messages and keep local autonomy when peers disappear; recover by exchanging changed data on reconnect.

LOCALIZATION / MERGE CONTRACT

- MVP: robots start at known common entry marker. Share initial relative transforms (not GPS) and translate graph coordinates.
- Add odometry drift + confidence. Refuse map merges when transform confidence or common reference is missing.
- Advanced: estimate transforms from overlapping map features; full collaborative SLAM is a later research milestone.

NO SUPERVISOR: PER-AGENT DECISION LOOP

LOCAL SENSE > **UPDATE GRAPH** > **READ PEERS**

SELECT GOAL > **ACT** > **BROADCAST DELTAS**

PASS GATE With 30% loss, agents still explore and recover after reconnect.

Integrate, validate, and demonstrate

STEP 5 bridges software to robots; STEP 6 proves robustness; schedule and primary references

STEP 5 ROS 2 + Gazebo

30 OCT

- Use Ubuntu 24.04 + ROS 2 Jazzy + Gazebo Harmonic; ros_gz_bridge connects sensors and robot commands.
- Spawn 2 simple rover models first; add 1 kinematic aerial proxy if integration time allows (not a full flight simulator).
- Per-robot ROS namespace: sensor -> local map / policy -> goal -> controller; DDS topics over LAN, not an ad hoc mesh itself.
- Rover: odom + lidar -> SLAM Toolbox -> Nav2 goal. Bridge /clock, /scan, /odom, /cmd_vel and tf as needed.

REUSE INFERENCE Same frontier feature encoder + goal policy; adapt only sensors / motion interfaces.

STEP 6 Evaluation + live demo

30-31 OCT

- Run unseen random maps; compare frontier baseline and RL on exactly the same 20 fixed evaluation seeds.
- Stress test 1 / 2 / 4 / 6 robots, changing maze complexity, false detection and comm loss of 0 / 10 / 30%.
- Log survivors found, coverage %, time, collisions, false alarms, messages and resilience after a disconnected robot.
- Live show: hidden survivor -> drone detection -> peer alert -> rover investigation -> joint map -> side-by-side metrics.

GO / NO-GO Keep 2D demo as primary; Gazebo only if stable and repeatable.

BUILD CALENDAR / FREEZE FUNCTIONAL MVP BEFORE FINAL DAY

26 OCT	27 OCT	28 OCT	29 OCT	30 OCT	31 OCT
Scope + API	World + sensors	Baseline + P2P	RL + tests	ROS optional + freeze	Live demonstration

DECISION GUARDRAILS

MUST: baseline + 4 agents + peer messages + measurable rescue demo

STRETCH: MAPPO critic, GNN, unknown-start map alignment, full PX4 drone flight dynamics, physical mesh.

VERIFIED TECHNICAL REFERENCES

01 PettingZoo Parallel API
02 RLlib multi-agent + PPO
03 Official MAPPO code
04 ROS / Gazebo compatibility
05 Gazebo ROS 2 bridge
06 SLAM Toolbox package
07 Nav2 Gazebo setup
08 Multi-robot map merge caveats